

**Development of a Reinforcement Learning Agent in Global Energy Markets using a
Continuous Training Architecture and MLOps Pipeline**

[Internal Project]

Caleb Kipchirchir

169391

ICS 4C

Supervisor Name

Allan Vikiru

**An Informatics and Computer Science Project Proposal Submitted to the School of
Computing and Engineering Sciences in Partial Fulfillment of the Requirements for the
Award of a Degree in Bachelor of Science in Informatics and Computer Science**

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

June 2026

Declaration and Approval

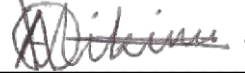
I declare that this work has not been previously submitted and approved for the award of a degree by this or any other University. To the best of my knowledge and belief, the research proposal contains no material previously published or written by another person except where due reference is made in the research proposal itself.

Student Name: Caleb Kipchirchir

Admission Number: 169391

Student Signature:  Date: **12/06/2026**

The Proposal of **Caleb Kipchirchir** has been reviewed and approved by **Mr. Allan Vikiru**

Supervisor Signature:  Date: **12/06/2026**

Acknowledgment

I express my sincere gratitude to my supervisor, Mr. Allan Vikiru, and my lecturer, Mr. Kelvin Omondi, for their invaluable guidance and technical expertise throughout the formulation of this proposal. Their constructive feedback was instrumental in sharpening the architectural focus of this MLOps pipeline. I am also deeply indebted to my family for their unwavering support and patience. Finally, a special thank you goes to my friends for providing the constant encouragement and motivation needed to keep the project moving forward.

Abstract

Global energy markets are highly volatile, increasing the demand for intelligent trading systems capable of adapting to rapidly changing conditions. Although Reinforcement Learning (RL) techniques such as Proximal Policy Optimization (PPO) have demonstrated effectiveness in developing risk-aware trading strategies, many existing solutions remain confined to controlled backtesting environments. As a result, these static models often experience performance degradation, commonly referred to as alpha decay, when exposed to live market dynamics and continuously evolving data. This study addresses this deployment challenge by designing and implementing an end-to-end Machine Learning Operations (MLOps) framework for the deployment, monitoring, and continuous adaptation of a PPO-based trading agent. Using an Object-Oriented Analysis and Design (OOAD) methodology, the proposed system integrates a DataOps pipeline for automated market data ingestion through the yfinance API, PostgreSQL-based data storage, a FinRL training environment, a FastAPI inference service, and a React.js visualization dashboard. A key contribution is the development of a Continuous Training (CT) orchestrator that monitors risk-adjusted performance metrics, particularly the Sharpe Ratio, to detect model drift. Upon identifying performance deterioration, the orchestrator automatically initiates retraining using updated market data and redeploys the optimized model. The study provides a scalable and reproducible architectural framework for resilient AI-driven trading systems operating in volatile financial environments.

Keywords: *Machine Learning Operations, Reinforcement Learning, Proximal Policy Optimization, Alpha Decay, Continuous Training, Model Drift*

Table of Contents

Declaration and Approval	ii
Acknowledgment	iii
Abstract	iv
List of figures	viii
List of tables	ix
List of abbreviations	x
Chapter 1: Introduction.....	1
1.1 Background Information	1
1.2 Problem Statement	2
1.3 Research Objectives	2
1.3.1 General Objective	2
1.3.2 Specific Objectives	2
1.4 Research Questions	3
1.5 Justification	3
1.6 Scope	3
1.7 Delimitations and Limitation	4
1.7.1 Delimitations	4
1.7.2 Limitation	4
Chapter 2: Literature Review.....	5
2.1 Introduction	5
2.2 Current State of Machine Learning in Algorithmic Trading	5
2.2.1 The State of Global Energy Markets and Algorithmic Trading	5
2.2.2 Available Machine Learning Architectures for Algorithmic Trading	5
2.3 Related Works in Automated Trading Systems	7
2.3.1 Hybrid Deep Learning Architectures for Financial Forecasting	7

2.3.2	A Systematic Approach to Portfolio Optimization	7
2.3.3	High Performance and Scalable Deep Reinforcement Learning for Quantitative Finance	7
2.3.4	Risk-Averse Policies for Natural Gas Futures Trading Using Distributional Reinforcement Learning	8
2.4	Gaps in Related Works.....	8
2.5	Conceptual Framework.....	9
Chapter 3:	Methodology	11
3.1	Introduction.....	11
3.2	Research Paradigm.....	11
3.2.1	Data Acquisition.....	11
3.2.2	Data Processing.....	12
3.2.3	Model Training.....	12
3.2.4	Model Validation and Testing	13
3.3	Development Approach	13
3.3.1	Prototype Planning and System Definition.....	14
3.3.2	Initial Prototype Development and Architectural Design.....	14
3.3.3	Agile Sprints	15
3.3.4	Testing and Validation.....	15
3.3.5	Final Prototype Evaluation	15
3.4	System Analysis Diagrams	16
3.4.1	Use Case Diagram.....	16
3.4.2	Class Diagram.....	16
3.4.3	Sequence Diagram	16
3.4.4	Activity Diagram	16
3.4.5	Package Diagram	17

3.5	System Design Diagrams	17
3.5.1	System Architecture Diagram	17
3.5.2	Wireframes	17
3.6	Tools and Techniques	18
3.6.1	FinRL and Stable Baselines3	18
3.6.2	Python Ecosystem (yfinance, Pandas, NumPy)	18
3.6.3	PostgreSQL	19
3.6.4	FastAPI	19
3.6.5	React.js	19
3.7	System Deliverables	19
3.7.1	System Documentation	20
3.7.2	Automated DataOps Pipeline	20
3.7.3	Trained PPO Trading Module	20
3.7.4	MLOps Continuous Training Orchestrator	20
3.7.5	Portfolio Visualization Dashboard	21
3.7.6	Deployment Package	21
3.7.7	System Viability and Responsiveness Report	21
	References	22
	Appendices	25

List of figures

Figure 2.5-1 End-to-End MLOps Conceptual Framework Diagram	10
Figure 3.3-1 Agile Software Development Life Cycle	14

List of tables

Table 3.1:Example of Enriched dataset Contents	12
--	----

List of abbreviations

ARIMA: AutoRegressive Integrated Moving Average

Bi-LSTM: Bidirectional Long Short-Term Memory

CNN: Convolutional Neural Network

CT: Continuous Training

DDPG: Deep Deterministic Policy Gradient

DQN: Deep Q-Network

DRL: Deep Reinforcement Learning

GARCH: Generalized Autoregressive Conditional Heteroskedasticity

GNN: Graph Neural Network

LLM: Large Language Model

LSTM: Long Short-Term Memory

MADRL: Multi-Agent Deep Reinforcement Learning

MDD: Maximum Drawdown

REST: Representational State Transfer

RLOps: Reinforcement Learning Operations

SAC: Soft Actor-Critic

SoC: Separation of Concerns

SPA: Single Page Application

UAT: User Acceptance Testing

UML: Unified Modeling Language

V&V: Verification and Validation

VIX: Volatility Index

Chapter 1: Introduction

1.1 Background Information

The financial markets have witnessed a paradigm shift with the integration of algorithmic trading, moving from manual execution to highly automated systems capable of processing vast amounts of market data in real-time (Autoriteit Consument & Markt (ACM), 2024). Within this landscape, the global energy sector presents a unique environment characterized by extreme volatility driven by macroeconomic shocks, geopolitical events, and the structural disruptions associated with the global transition to green energy (International Renewable Energy Agency (IRENA), 2024).

Historically, the application of machine learning in these markets has been dominated by predictive modeling, utilizing standard neural networks to forecast future price movements (Huang et al., 2026). While these models can achieve high accuracy in forecasting, they leave the critical step of execution—deciding when to buy, hold, or sell—to rigid, rule-based wrappers. Consequently, the financial industry has recognized a necessary shift toward "Agentic AI," replacing passive predictive models with goal-oriented, autonomous agents. Reinforcement Learning (RL), specifically the Proximal Policy Optimization (PPO) algorithm, has emerged as a superior alternative for continuous action spaces, shifting the focus from predictive forecasting to prescriptive, risk-averse execution (Espiga-Fernández et al., 2024).

Despite the theoretical promise of Agentic AI, a massive engineering hurdle known as the "Deployment Chasm" remains. Current academic literature heavily emphasizes model training in isolated, static simulated environments, largely ignoring the critical software infrastructure required to actually deploy these models into live applications (Xia et al., 2026). As artificial intelligence adoption rises, static trading algorithms suffer from rapid alpha decay—a phenomenon where algorithmic homogenization and reflexive signal erosion severely compress excess returns (Meng & Chen, 2026). To survive market drift, autonomous agents require end-to-end Machine Learning Operations (MLOps) pipelines designed to continuously ingest real-time data, evaluate agent performance, and automatically retrain neural network weights without human intervention (Li et al., 2021a).

1.2 Problem Statement

Ideally, Agentic AI systems should seamlessly bridge the gap between complex mathematical reinforcement learning and live-market execution, autonomously adapting to new data as market regimes shift. However, the reality is that quantitative analysts face a severe lack of reproducible deployment infrastructure. While sophisticated Proximal Policy Optimization (PPO) models can be trained in sterile environments, they consistently fail in production due to a lack of automated DataOps pipelines and the inability to handle concept drift.

Without a closed-loop system to continuously monitor model health and automatically trigger retraining, static AI agents fall victim to alpha decay and become obsolete. Furthermore, "black box" reinforcement learning models are difficult for stakeholders to interpret without integrated visualization tools. This study proposes solving this infrastructure bottleneck by engineering an end-to-end Machine Learning Operations (MLOps) architecture—combining PostgreSQL, FinRL, FastAPI, and React.js—to deploy a PPO trading agent wrapped in a Continuous Training (CT) loop specifically designed to detect alpha decay and autonomously retrain the model on global energy equities.

1.3 Research Objectives

1.3.1 General Objective

To design, develop, and deploy an automated end-to-end MLOps pipeline and Continuous Training (CT) architecture for a Reinforcement Learning trading agent in global energy markets.

1.3.2 Specific Objectives

- i. To analyse the current state of Agentic AI deployment and the architectural limitations of static machine learning models in algorithmic trading.
- ii. To design an automated DataOps pipeline capable of continuous historical and daily financial data ingestion, normalization, and database storage.
- iii. To develop a Proximal Policy Optimization (PPO) agent within a simulated trading environment to execute autonomous, risk-averse trading actions.
- iv. To test the system's architectural reliability alongside simulated baseline financial performance metrics.

1.4 Research Questions

- i. What is the current state of Agentic AI deployment, and what are the primary architectural limitations of static machine learning models in algorithmic trading?
- ii. How can an automated DataOps pipeline be designed and implemented to handle continuous historical and daily financial data ingestion, normalization, and database storage?
- iii. How effectively can a Proximal Policy Optimization (PPO) agent be developed within a simulated environment to execute autonomous, risk-averse trading actions?
- iv. How does the developed system perform when evaluated against metrics of architectural reliability alongside simulated baseline financial performance indicators?

1.5 Justification

Current algorithmic trading research predominantly focuses on the mathematics of price prediction within static environments (Huang et al., 2026). The major gap identified across modern literature is the lack of systems that specialize in scalable deployment architectures, commonly referred to as "RLOps" or MLOps (Banka, 2025; Li et al., 2021a).

By engineering a complete, closed-loop software architecture where a relational database, an RL training environment, a FastAPI backend, and a React frontend communicate autonomously, this project shifts the academic focus from isolated market prediction to vital system deployment. This work heavily benefits the fields of Informatics and Computer Science by providing a reproducible blueprint for deploying continuously adapting artificial intelligence, solving the infrastructure bottleneck that currently prevents Agentic AI from surviving in dynamic environments.

1.6 Scope

This study focuses on developing the software architecture required to deploy an autonomous trading agent. From a technical perspective, the project entails the development of an automated Machine Learning Operations (MLOps) pipeline. This pipeline utilizes PostgreSQL for robust time-series data storage, the FinRL and Stable Baselines3 libraries for training the Proximal Policy Optimization (PPO) agent, FastAPI for efficient backend model serving, and React.js to construct the interactive visualization dashboard.

Regarding the data scope, the system is designed for the programmatic ingestion of both historical and daily Open, High, Low, Close, and Volume (OHLCV) data. This continuous data stream targets top-tier global energy equities, such as ExxonMobil and Shell, securely sourced and updated via the yfinance Application Programming Interface (API).

Functionally, the scope encompasses the implementation of a Continuous Training (CT) orchestrator. This operational module is specifically engineered to continuously monitor the portfolio's performance metrics, primarily focusing on Sharpe Ratios and automatically trigger background retraining cycles when the system detects signal erosion or market drift.

1.7 Delimitations and Limitation

1.7.1 Delimitations

This study deliberately focuses solely on the software engineering architecture applied to the global energy equity sector; it excludes cryptocurrency, forex, and high-frequency arbitrage to ensure the infrastructure is tested on a defined set of market mechanics

1.7.2 Limitation

A primary limitation of this study lies within the "Sim2Real" execution gap. While simulated trading environments are essential for backtesting reinforcement learning agents, they cannot perfectly replicate the complex mechanics of live financial markets. Specifically, these simulations often fail to accurately account for real-world transaction costs, execution slippage, and strict liquidity constraints(Xia et al., 2026). Consequently, the primary evaluation of this system focuses on its architectural robustness and software reliability—proving the viability of the deployment infrastructure—rather than guaranteeing live-market financial profit.

Furthermore, the system faces distinct algorithmic limitations. Deep Reinforcement Learning algorithms, such as Proximal Policy Optimization (PPO), are inherently sample-inefficient. Because this study relies on daily OHLCV data rather than high-frequency, institutional tick-level data, the agent's observation space remains limited. This data constraint may lead to localized overfitting, particularly when the agent encounters simulated market crashes.

Chapter 2: Literature Review

2.1 Introduction

This chapter surveys and critically evaluates the existing scholarly literature relevant to the application of Deep Reinforcement Learning (DRL) and Machine Learning Operations (MLOps) in autonomous algorithmic trading. To align with the architectural focus of this study, the review evaluates prior research through a software engineering and deployment lens. It is structured around four foundational pillars: the evolution and limitations of traditional predictive models; the paradigm shift toward Agentic AI; the critical engineering requirement of Continuous Training (CT) MLOps pipelines; and the mathematical reality of AI-driven alpha decay, which necessitates automated retraining architectures within hyper-volatile energy markets.

2.2 Current State of Machine Learning in Algorithmic Trading

2.2.1 The State of Global Energy Markets and Algorithmic Trading

To understand the necessity of autonomous trading agents, one must first define the operational environment. The global energy sector presents a highly unique, hyper-volatile financial landscape driven by exogenous macroeconomic shocks, geopolitical conflicts, and the structural disruptions associated with the global transition to green energy. Trading in this context refers to the systematic execution of buy, hold, or sell orders on energy equities (e.g., ExxonMobil, Shell) and commodity derivatives to manage risk and capture excess returns (alpha). Historically, trading was a manual, human-driven process dependent on fundamental analysis. However, the sheer velocity and volume of modern market data have forced a paradigm shift. Today, trading is fundamentally a computational problem, requiring highly automated algorithmic systems capable of processing real-time time-series data, modeling complex market mechanics, and executing orders at speeds far beyond human cognitive capacity.

2.2.2 Available Machine Learning Architectures for Algorithmic Trading

As algorithmic trading has evolved, quantitative analysts have leveraged a diverse spectrum of machine learning architectures to model market behaviour, with each generation of models addressing limitations of its predecessors. Early automation relied heavily on statistical and linear models such as ARIMA (Autoregressive Integrated Moving Average) and GARCH (Generalized Autoregressive Conditional Heteroskedasticity). These techniques provided useful baselines for

forecasting trends and modeling volatility, but they were built on assumptions of linear relationships and relatively stable market structures. As a result, they often struggled to capture the highly non-stationary, noisy, and complex dynamics characteristic of financial time-series data.

To overcome these limitations, the industry increasingly adopted deep learning architectures designed for sequential data. Long Short-Term Memory (LSTM) networks and Bidirectional LSTMs (Bi-LSTMs) became particularly popular because of their ability to learn non-linear temporal dependencies and long-range patterns from historical market information. By retaining information over extended sequences, these models improved the accuracy of price forecasting and enabled more sophisticated analyses of market movements.

In parallel, Convolutional Neural Networks (CNNs), originally developed for computer vision applications, found new uses in quantitative finance. Rather than processing images alone, CNNs were adapted to analyse structured market representations such as limit order books, candlestick charts, and technical indicator matrices. Their strength lies in extracting localized spatial features and identifying subtle micro-patterns that may precede price breakouts or shifts in market behaviour.

Despite the predictive power of LSTMs and CNNs, these architectures remain fundamentally forecasting tools. They can estimate future price movements with high statistical accuracy, but they do not inherently determine how a trading system should act on those predictions. In practice, their outputs typically require additional rule-based logic to generate buy, sell, or hold decisions. This separation between prediction and execution has motivated a growing shift toward Reinforcement Learning (RL) approaches.

Modern RL architectures, including Deep Q-Networks (DQN), Soft Actor-Critic (SAC), and Proximal Policy Optimization (PPO), take a fundamentally different approach. Rather than focusing solely on forecasting future prices, these models learn optimal trading and execution policies through direct interaction with simulated market environments. By continuously evaluating rewards, penalties, and risk-adjusted outcomes, RL agents can make sequential decisions that balance profitability and risk management. Consequently, the industry's focus is increasingly moving from passive prediction toward agentic, goal-oriented systems that learn to act autonomously in dynamic market conditions.

2.3 Related Works in Automated Trading Systems

2.3.1 Hybrid Deep Learning Architectures for Financial Forecasting

In their investigation of hybrid deep learning architectures, (Kreuzberger et al., 2023) assessed whether combining deep learning models with statistical baselines could outperform standalone architectures in financial forecasting. Their methodology utilized the independent training of an Autoregressive Integrated Moving Average (ARIMA) model and an Attention-enhanced Long Short-Term Memory (LSTM) network, followed by ensemble aggregation. While the study demonstrated superior regression performance, the proposed model assumed frictionless trading. Crucially, the authors highlighted a fundamental barrier inherent to supervised learning, noting that predictive accuracy does not uniformly translate to improved economic returns in live market environments.

2.3.2 A Systematic Approach to Portfolio Optimization

Exploring systematic portfolio optimization, (Espiga-Fernández et al., 2024) conducted an empirical comparative study evaluating various continuous and discrete-action Deep Reinforcement Learning (DRL) agents across diverse market indices, including the S&P 500. The researchers applied DQN, DDPG, PPO, and SAC algorithms, utilizing Convolutional Neural Network (CNN) feature extractors. The study found that PPO significantly outperformed vanilla Multi-Layer Perceptrons (MLPs) in terms of risk efficiency. However, a major limitation of this research is its assumption of highly liquid fractional asset trading, which fails to account for the specific nuances and localized volatility characteristic of the energy sector.

2.3.3 High Performance and Scalable Deep Reinforcement Learning for Quantitative Finance

Focusing on deployment infrastructure, (Li et al., 2021) formulated stock trend prediction as a Markov Decision Process (MDP) and proposed an "RLOps in finance" paradigm. By deploying a cloud-native architecture using NVIDIA DGX SuperPOD infrastructure, they accelerated DRL strategy development, training models such as PPO, DDPG, and SAC over a 10-year span. While this framework provides a definitive blueprint for constructing scalable MLOps pipelines, it focuses heavily on large-scale broad equity tracking. Consequently, it has not been

comprehensively adapted to handle the complex derivative structures and sudden shocks found in global energy markets.

2.3.4 Risk-Averse Policies for Natural Gas Futures Trading Using Distributional Reinforcement Learning

Addressing the unique challenges of the energy sector, (Hêche et al., 2025) presented a specialized study demonstrating that Distributional Reinforcement Learning can construct highly adaptable, risk-sensitive trading policies for natural gas futures. Their methodology applied advanced algorithms such as C51, IQN, and QR-DQN, which were rigorously tested against proprietary historical datasets. Although the study successfully proves the necessity of explicit risk-averse policy generation when dealing with severe energy market volatility, its scope remains narrowly focused on a single commodity, thereby failing to address broader cross-commodity hedging requirements.

2.4 Gaps in Related Works

A critical synthesis of the reviewed literature reveals persistent software and infrastructure limitations in current algorithmic trading research, which this study aims to directly address. Foremost among these is the "Sim2Real" deployment chasm. As identified by recent audits (Xia et al., 2026) the vast majority of Deep Reinforcement Learning (DRL) trading agents exist merely as academic prototypes evaluated in static backtesting environments. Modern literature frequently omits the rigorous software engineering required to deploy these models into live architectures, such as FastAPI or React, that can handle continuous data streaming.

Furthermore, there is a distinct lack of automated DataOps and Continuous Training (CT) infrastructure. While frameworks like FinRL successfully provide the necessary simulation environments for agent development (Li et al., 2021a), researchers typically rely on static CSV files for model training. Consequently, there is a glaring absence of integrated DataOps pipelines capable of programmatically fetching, cleaning, and storing financial data to feed the artificial intelligence continuously in a production setting.

Finally, this lack of infrastructure leaves a critical vulnerability: unmitigated alpha decay within hyper-volatile energy markets. While the Proximal Policy Optimization (PPO) algorithm has proven effective in managing energy volatility (Verma & Xie, 2025), static models are

mathematically proven to fail long-term due to AI-driven signal erosion (Meng & Chen, 2026). Current research lacks closed-loop Continuous Training orchestrators specifically designed to continuously monitor risk-adjusted metrics and autonomously trigger background retraining, which is an absolute requirement for an agent to survive market drift

2.5 Conceptual Framework

The conceptual framework of this study models an automated end-to-end Machine Learning Operations (MLOps) pipeline designed to bridge the deployment chasm in algorithmic trading. The system's inputs consist of continuous historical and real-time OHLCV data from global energy equities, securely ingested via the yfinance API. This raw data flows into a DataOps processing layer where it is programmatically cleaned, normalized via rolling Z-score standardization, and persisted in a PostgreSQL time-series database. The core processing variable is the Proximal Policy Optimization (PPO) agent, which actively trains its execution policy within a FinRL simulated environment using the ingested historical data.

The primary outputs of the system are the agent's autonomous, discrete trading actions (buy, hold, sell) and real-time risk-adjusted portfolio metrics, such as the Sharpe Ratio and Maximum Drawdown, which are surfaced via a React.js presentation layer. Crucially, the framework is governed by a cyclical feedback relationship: a Continuous Training (CT) orchestrator constantly monitors the output telemetry. If it detects performance degradation or market drift, it autonomously triggers the processing layer to fetch fresh data, retrain the PPO agent, and redeploy updated model artifacts via the FastAPI backend, ensuring sustained architectural reliability without human intervention.

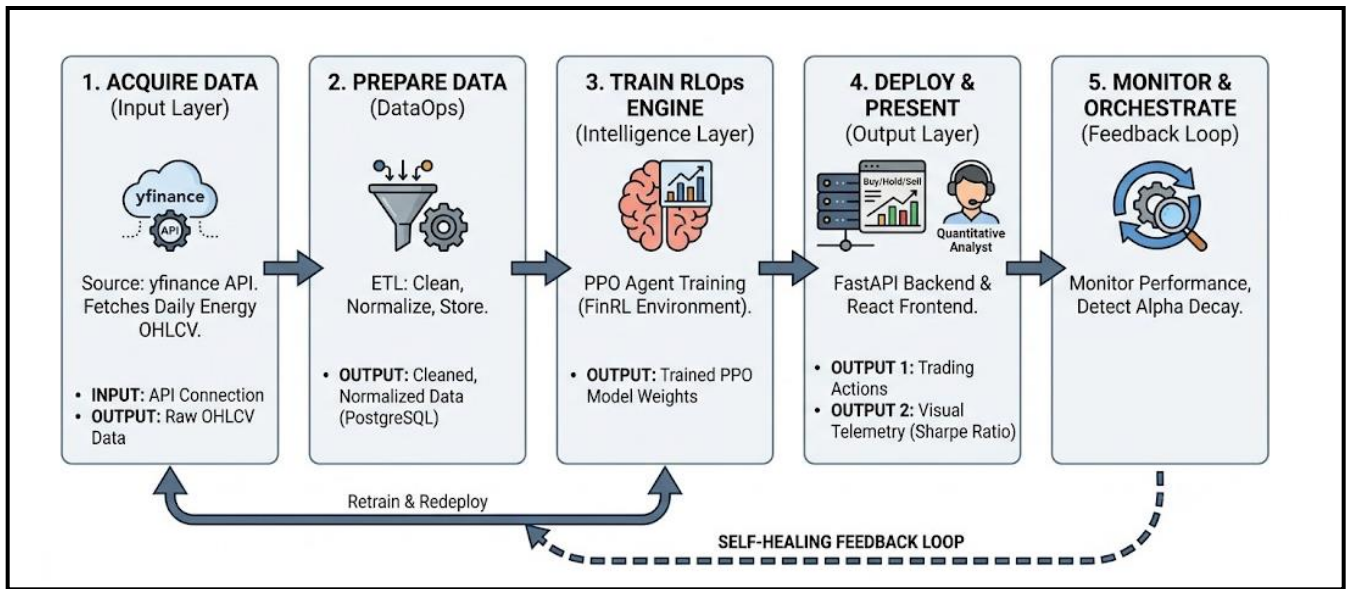


Figure 2.5-1 End-to-End MLOps Conceptual Framework Diagram

Chapter 3: Methodology

3.1 Introduction

This chapter outlines the systematic methodological approach adopted for the development of the autonomous trading system. Building an algorithmic trading platform integrated with a Machine Learning Operations (MLOps) pipeline requires a hybrid methodological framework that bridges quantitative financial data science with rigorous software engineering (Kreuzberger et al., 2023a). Consequently, this study adopts an Experimental Research Paradigm to govern the machine learning pipeline, complemented by a Design Thinking methodology wrapped in Agile Sprints for the software development lifecycle.

To satisfy the structural engineering requirements of the system, this project explicitly adopts the Object-Oriented Analysis and Design (OOAD) development paradigm. OOAD principles are utilized to model the system's static structures and dynamic behavioural blueprints. This chapter details the data handling processes, the iterative software development approach, the Unified Modeling Language (UML) diagrams for system analysis, and the specific technological stack and deliverables that encompass the project (Paetzold - Byhain & Scharold, 2024).

3.2 Research Paradigm

To ensure the scientific validity and architectural reliability of the trading agent, the study employs an experimental research paradigm tailored for quantitative finance. This approach enables iterative experimentation, empirical measurement of algorithmic performance against shifting market regimes, and the systematic tuning of the Deep Reinforcement Learning (DRL) agent before live deployment.

3.2.1 Data Acquisition

The foundation of the reinforcement learning module relies on programmatic data ingestion. Data will be sourced programmatically using the yfinance API to extract daily financial metrics for top-tier global energy equities (for example., ExxonMobil, Shell, Chevron). The raw data will be enriched with technical indicators during the DataOps processing phase to expand the Proximal Policy Optimization (PPO) agent's observation space, as seen below in Table 3.1 .

Table 3.1: Example of Enriched dataset Contents

Date	Ticker	Close Price (\$)	Volume	SMA_20	RSI_14	VIX (Volatility)
2025-06-01	XOM	115.50	15,400,000	112.30	65.2	18.5
2025-06-02	XOM	114.20	18,200,000	112.45	60.1	19.2
2025-06-01	SHEL	72.80	8,100,000	70.10	58.4	18.5
2025-06-02	SHEL	71.50	9,500,000	70.25	54.2	19.2

3.2.2 Data Processing

Financial time-series data is inherently noisy, necessitating a rigorous DataOps preprocessing pipeline. The data processing phase will involve handling missing daily ticks through forward-fill imputation to prevent sequence breakage in the agent’s state observations. The data will be mathematically adjusted for stock splits and dividend distributions to ensure price continuity. Finally, all numeric features will undergo rolling Z-score standardization to normalize varying nominal price levels, ensuring the neural network learns generalized market mechanics. The processed dataset will then be stored in a PostgreSQL database and partitioned into chronological training and out-of-sample backtesting sets.

3.2.3 Model Training

Model training in this study involves establishing a Markov Decision Process (MDP) within a simulated financial environment (FinRL). The Proximal Policy Optimization (PPO) algorithm will be trained to act as the primary Agentic AI. The agent’s state space (market observations) and action space (continuous or discrete buy, hold, and sell weights) will be explicitly defined. Model training will utilize an engineered reward function designed to penalize Maximum Drawdown, forcing the agent to learn risk-averse execution policies. Hyperparameter tuning will optimize the PPO agent’s learning rate and discount factor, balancing sample efficiency with stable policy updates.

3.2.4 Model Validation and Testing

The trained PPO model will be rigorously evaluated against unseen, out-of-sample historical data representing distinct market regimes. Crucially, validation is twofold. First, financial viability is measured using institutional quantitative metrics, including Cumulative Return, the Sharpe Ratio, and Maximum Drawdown. Second, and more importantly for this software architecture, Pipeline Responsiveness will be tested. This involves empirically validating the Continuous Training (CT) loop's ability to detect a drop in the Sharpe Ratio and successfully trigger a background model retraining cycle without crashing the backend server.

3.3 Development Approach

To successfully translate the theoretical Deep Reinforcement Learning (DRL) mathematics into a highly available, end-to-end MLOps production environment, this study adopts the Agile Software Development Life Cycle (SDLC). Traditional deterministic development models such as Waterfall are fundamentally incompatible with artificial intelligence systems because machine learning is inherently stochastic and requires continuous experimental tuning (Amershi et al., 2019). Agile is explicitly selected because its iterative framework accommodates the rigorous trial-and-error nature of hyperparameter optimization. It allows for the continuous integration, backtesting, and refinement of the Proximal Policy Optimization (PPO) agent across defined development cycles without halting the overarching software engineering pipeline (Kreuzberger et al., 2023b). The cyclical progression of these development sprints is visually summarized in Figure 3.3-1 below.

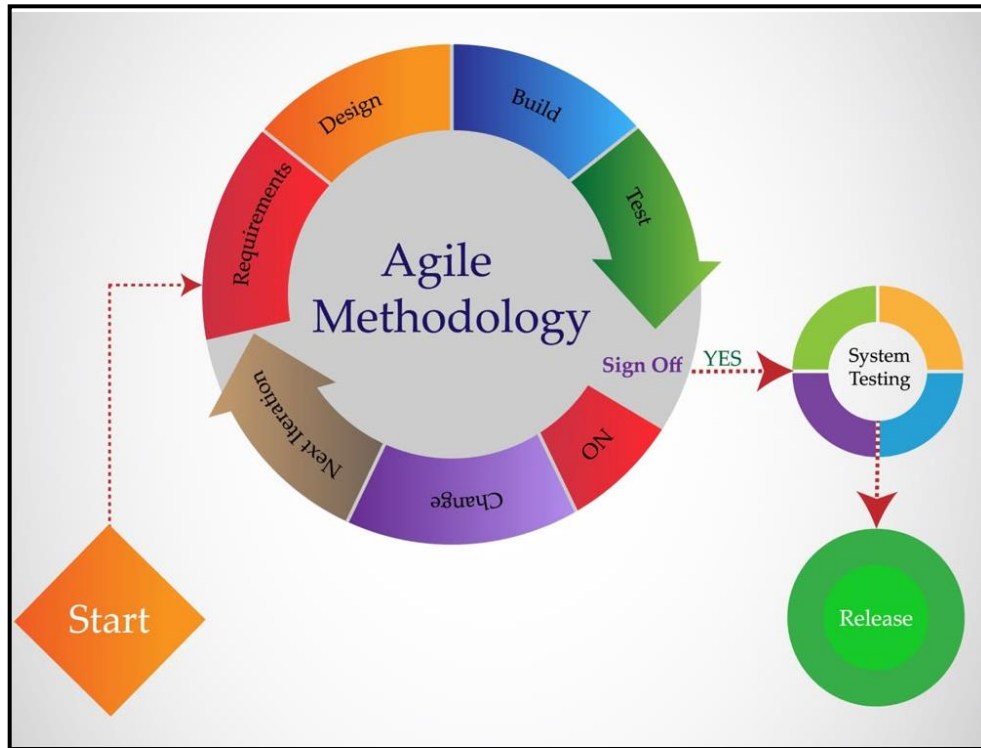


Figure 3.3-1 Agile Software Development Life Cycle

3.3.1 Prototype Planning and System Definition

In Agile prototyping, the initial step involves delineating the strict functional, non-functional, and data-centric requirements of the MLOps platform. Functionally, the system must autonomously ingest daily OHLCV time-series data and execute a Continuous Training (CT) orchestrator capable of empirically detecting model drift (Lu et al., 2018). Non-functionally, the architecture must guarantee high availability for the DataOps pipelines, ensure fault tolerance during background model weight updates, and expose a low-latency web interface for real-time portfolio risk monitoring.

3.3.2 Initial Prototype Development and Architectural Design

During this phase, the first version of the system architecture is established. The design rigorously enforces the Object-Oriented Analysis and Design (OOAD) paradigm, adopting a microservices-inspired approach. Because training a deep reinforcement learning agent is highly computationally intensive, this architecture ensures the learning logic is strictly decoupled from the data ingestion and presentation layers. The development stack utilizes PostgreSQL for persistent time-series

storage, FastAPI for a stateless inference backend, and React.js for a component-driven client interface.

3.3.3 Agile Sprints

The software will be engineered and refined iteratively through four time-boxed Agile sprints, with each cycle involving testing and stakeholder evaluation. Sprint 1 establishes the DataOps foundation by developing programmatic yfinance API scrapers and structuring the PostgreSQL database for the automated data ingestion pipeline. Following this, Sprint 2 centers on environment configuration by setting up the FinRL simulated trading environment and defining the strict state and continuous action spaces required to model the global energy sector as a Markov Decision Process (MDP) (Li et al., 2021). Sprint 3 then shifts to RLOps and model training, focusing heavily on the core AI by training the Proximal Policy Optimization (PPO) agent and engineering a risk-averse reward function explicitly designed to penalize maximum drawdown. Finally, Sprint 4 handles deployment and orchestration, unifying the system into a deployable platform by wrapping the trained model in a FastAPI backend, constructing the React.js dashboard, and cementing the automated Continuous Training (CT) feedback loop.

3.3.4 Testing and Validation

Each prototype sprint will conclude with a rigorous Verification and Validation phase to ensure architectural integrity. Standard software unit testing is insufficient for MLOps; therefore, the model validation will utilize walk-forward empirical backtesting using out-of-sample historical market data to mitigate the "Sim2Real" execution gap and prevent algorithmic overfitting (Bailey et al., 2014). Concurrently, automated integration testing will evaluate the FastAPI backend, verifying the stateless execution of the model endpoints during the automated retraining cycles.

3.3.5 Final Prototype Evaluation

In this final phase, the completed MLOps deployment package will be evaluated against the core project objectives. Success will be measured by verifying the system's ability to seamlessly ingest live data, execute autonomous trades, and successfully trigger background retraining upon detecting performance degradation, proving the architecture is ready for live-market simulated scaling.

3.4 System Analysis Diagrams

As mandated by the chosen development paradigm, Object-Oriented Analysis and Design (OOAD) principles are utilized to map the system's logic. The following Unified Modeling Language (UML) diagrams provide a visual blueprint of the system's architecture.

3.4.1 Use Case Diagram

The Use Case Diagram defines the system boundaries and illustrates the interactions between external actors (for example, Quantitative Analyst, Market Data API) and the core functionalities of the system. It maps use cases such as ingesting market data, executing a trade action, monitoring alpha decay, and triggering continuous retraining.

3.4.2 Class Diagram

The Class Diagram models the static object-oriented structure of the backend. It details the primary system classes (for example, DataIngestor, PPOAgent, EnvironmentSim, MLOpsOrchestrator), their encapsulated attributes (for example, `policy_weights`, `target_sharpe_threshold`), and operational methods (for example, `predict_action ()`, `evaluate_model_drift ()`), alongside their multiplicity relationships.

3.4.3 Sequence Diagram

To model dynamic behaviour, the Sequence Diagram illustrates the chronological flow of messages between objects during the Continuous Training (CT) cycle. It maps the timeline of events from the moment the orchestrator detects alpha decay, detailing how fresh data is requested from the database, how the PPO agent updates its policy, and how the new model artifacts are saved and served.

3.4.4 Activity Diagram

The Activity Diagram provides a flowchart representation of the system's internal control flows and decision-making algorithms. It captures the logical branching of a localized trading cycle, illustrating the exact mathematical threshold conditions under which the system decides to execute a discrete trade (Buy/Hold/Sell). Crucially, it maps the fail-safe constraints: if macroeconomic volatility (for example, the VIX index) breaches a predefined threshold, the activity flow bypasses the trading execution node and routes directly to an emergency capital-preservation state.

3.4.5 Package Diagram

The Package Diagram organizes the complex codebase into high-level logical modules, enforcing the Principle of Separation of Concerns (SoC). The system is divided into distinct namespaces: DataOps (handling ETL pipelines and PostgreSQL bridging), RLOps (containing FinRL and Stable Baselines3 model logic), and Serving (FastAPI and routing). This diagram demonstrates strict architectural modularity, ensuring high cohesion within modules and low coupling across the system network.

3.5 System Design Diagrams

3.5.1 System Architecture Diagram

The System Architecture Diagram provides a visual representation of the platform's layered and loosely coupled architecture within the end-to-end MLOps deployment ecosystem. It illustrates the complete flow of data and asynchronous processes across four key infrastructure layers: the DataOps Layer, responsible for extracting, transforming, and loading financial data from the yfinance API into a PostgreSQL database; the RLOps Training Layer, which contains the FinRL training environment and automated model retraining orchestrator; the Serving Layer, powered by a stateless FastAPI backend that efficiently handles concurrent inference requests; and the Presentation Layer, implemented as a React.js Single Page Application for end-user interaction. Furthermore, the diagram highlights the system's closed-loop feedback architecture, demonstrating how performance monitoring and telemetry data automatically initiate model retraining workflows while ensuring uninterrupted operation of the live inference services.

3.5.2 Wireframes

The wireframes function as the foundational design blueprints for the application's Graphical User Interface (GUI), providing a structured representation of the dashboard before implementation. Beyond serving as basic visual mockups, they incorporate Human–Computer Interaction (HCI) principles to enhance usability, information accessibility, and overall user experience within the React.js-based platform. The wireframes define the layout, navigation structure, and component organization required to effectively present complex financial and operational data. Key interface elements include designated areas for real-time portfolio performance visualization through cumulative equity curves, tabular displays with pagination for autonomous trading activity logs

(Buy, Hold, and Sell decisions), and dedicated panels for system observability. In alignment with the platform's MLOps-driven architecture, the wireframes also incorporate a specialized monitoring dashboard that provides visibility into model performance metrics, including real-time Sharpe Ratio monitoring, performance degradation alerts, and the operational status of the Continuous Training (CT) pipeline.

3.6 Tools and Techniques

The following technological stack has been carefully selected to ensure high performance, mathematical rigor, and seamless integration between quantitative finance and software engineering workflows. The selection criteria heavily prioritized architectural resilience, asynchronous processing capabilities, and the strict decoupling of data engineering from artificial intelligence inference.

3.6.1 FinRL and Stable Baselines3

The FinRL open-source library will be utilized as the foundational simulated trading environment, mathematically converting historical energy market data into standard OpenAI Gym Markov Decision Process (MDP) formats. To handle the neural network execution, Stable Baselines3 provides a highly optimized, PyTorch-backed implementation of the Proximal Policy Optimization (PPO) architecture. Utilizing Stable Baselines3 ensures that the Actor-Critic network updates are mathematically stable and computationally efficient, providing a robust, production-ready foundation rather than relying on brittle, custom-built algorithmic implementations.

3.6.2 Python Ecosystem (yfinance, Pandas, NumPy)

The core orchestration logic and DataOps pipelines are engineered entirely within the Python ecosystem. The yfinance library acts as the programmatic API contract, providing secure, continuous access to global market data. To handle the intense computational load of preprocessing, Pandas and NumPy will be utilized as vectorized data manipulation engines. By leveraging optimized C-level arrays beneath the Python interface, these libraries bypass standard execution bottlenecks, ensuring that complex tasks—such as missing value imputation and high-frequency rolling Z-score standardizations—are executed with minimal latency before data enters the neural network.

3.6.3 PostgreSQL

PostgreSQL has been selected as the primary relational database management system to anchor the system's persistent storage. Far beyond standard data logging, it acts as the ACID-compliant backbone for the Continuous Training (CT) loop. Its robust capability to handle structured, high-volume time-series financial data ensures that the continuous ingestion required for the MLOps pipeline remains highly scalable. Furthermore, proper indexing strategies within PostgreSQL will guarantee that the multi-threaded retraining orchestrator can fetch large historical batches of energy market ticks without degrading the database's read/write performance.

3.6.4 FastAPI

FastAPI will serve as the highly concurrent backend framework for model deployment and orchestration. It has been selected due to its asynchronous Server Gateway Interface (ASGI) architecture, which prevents the computationally heavy Reinforcement Learning background loops from blocking live inference requests. Additionally, FastAPI heavily utilizes Pydantic for strict data validation and serialization. This ensures that any incoming financial data structures strictly adhere to expected tensor formats before reaching the PPO agent, effectively preventing malformed market data from causing critical system failures during autonomous execution.

3.6.5 React.js

React.js has been chosen as the frontend framework to construct the quantitative visualization dashboard, enforcing a strict Separation of Concerns (SoC) between the backend AI logic and the client-side presentation layer. Its component-based architecture and Virtual DOM rendering are essential for processing high-frequency data streams. This guarantees that highly dynamic portfolio telemetry, localized maximum drawdown gauges, and real-time trading charts can be rendered and updated seamlessly without inducing browser latency or memory leaks on the client's machine.

3.7 System Deliverables

Upon successful completion of the development lifecycle, the project will produce a comprehensive, production-ready algorithmic trading platform that effectively bridges the "Deployment Chasm" by integrating advanced reinforcement learning, automated data

engineering, continuous model management, and user-centric visualization capabilities. The key deliverables are outlined below:

3.7.1 System Documentation

A comprehensive technical documentation suite detailing the system architecture, software design specifications, UML diagrams, database schemas, API documentation, deployment procedures, and maintenance guidelines. The documentation will also include a detailed quantitative evaluation report presenting backtesting results, risk-adjusted performance metrics, hyperparameter configurations, and comparative analyses against benchmark trading strategies.

3.7.2 Automated DataOps Pipeline

A robust and fully automated data engineering pipeline capable of continuously acquiring, validating, cleansing, transforming, and storing financial market data from multiple sources. The pipeline will ensure data integrity, consistency, and availability while supporting scalable processing of historical and real-time market information required for model training and inference.

3.7.3 Trained PPO Trading Module

A production-ready Proximal Policy Optimization (PPO) reinforcement learning engine trained on historical market data and optimized to maximize long-term risk-adjusted returns. The module will incorporate market state representation, reward engineering, transaction cost modelling, and portfolio risk considerations, enabling intelligent trading decisions under dynamic market conditions.

3.7.4 MLOps Continuous Training Orchestrator

An intelligent MLOps framework responsible for automated model lifecycle management. The orchestrator will continuously monitor model performance using key metrics such as Sharpe Ratio, Sortino Ratio, maximum drawdown, and cumulative returns. Upon detecting performance degradation, market regime shifts, or concept drift, the system will automatically trigger retraining, validation, versioning, and redeployment workflows to maintain model effectiveness.

3.7.5 Portfolio Visualization Dashboard

A modern, interactive React-based analytics dashboard providing stakeholders with real-time visibility into portfolio performance and system operations. The dashboard will feature live equity curves, profit and loss tracking, portfolio allocation breakdowns, drawdown analysis, risk metrics, trading activity logs, model performance indicators, and historical strategy comparisons through intuitive visualizations.

3.7.6 Deployment Package

A fully containerized and deployment-ready software package incorporating the backend inference services, database infrastructure, frontend dashboard, DataOps pipeline, and MLOps components. The package will support reproducible deployments across cloud and on-premises environments through Docker-based containerization and configuration management, ensuring scalability, maintainability, and operational reliability.

3.7.7 System Viability and Responsiveness Report

A comprehensive assessment report documenting the architectural effectiveness of the MLOps pipeline. While baseline financial metrics (for example, Sharpe Ratio) will be tracked, the primary objective of this evaluation is not to guarantee absolute profitability, but rather to provide empirical evidence of the system's architectural responsiveness. The report will validate the Continuous Training (CT) loop's ability to successfully detect simulated market drift, trigger a background retraining cycle, and deploy updated weights without pipeline failure or server downtime.

References

- Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software Engineering for Machine Learning: A Case Study. *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>
- Autoriteit Consument & Markt (ACM). (2024). *Algorithmic trading in wholesale energy markets: Market study on uses, trends, and regulatory impacts* (Case No. ACM/23/184887). https://www.acm.nl/system/files/documents/rapport-marktstudie-algoritmische-handel-energiemarkt-public_0.pdf
- Bailey, D. H., Borwein, J. M., López De Prado, M., & Zhu, Q. J. (2014). Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance. *Notices of the American Mathematical Society*, 61(5), 458. <https://doi.org/10.1090/noti1105>
- Banka, T. R. S. B. (2025). *Agentic AI and Multimodal AI to Quantitative Finance using Algorithmic Trading for Wealth & Portfolio Management* [Dissertation]. <https://repository.learn-portal.org/index.php/rps/article/view/998>
- Espiga-Fernández, F., García-Sánchez, Á., & Ordieres-Meré, J. (2024). A Systematic Approach to Portfolio Optimization: A Comparative Study of Reinforcement Learning Agents, Market Signals, and Investment Horizons. *Algorithms*, 17(12), 570. <https://doi.org/10.3390/a17120570>
- Hêche, F., Nigro, B., Barakat, O., & Robert-Nicoud, S. (2025). *Risk-averse policies for natural gas futures trading using distributional reinforcement learning* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2501.04421>
- Huang, M., Liu, J., Wu, X., Sun, X., Zhuang, W., & Chen, Z. (2026). A fairness-aware framework for value sharing in local electricity markets using graph and sequence learning with multi-agent reinforcement. *PeerJ Computer Science*, 12, e3774. <https://doi.org/10.7717/peerj-cs.3774>

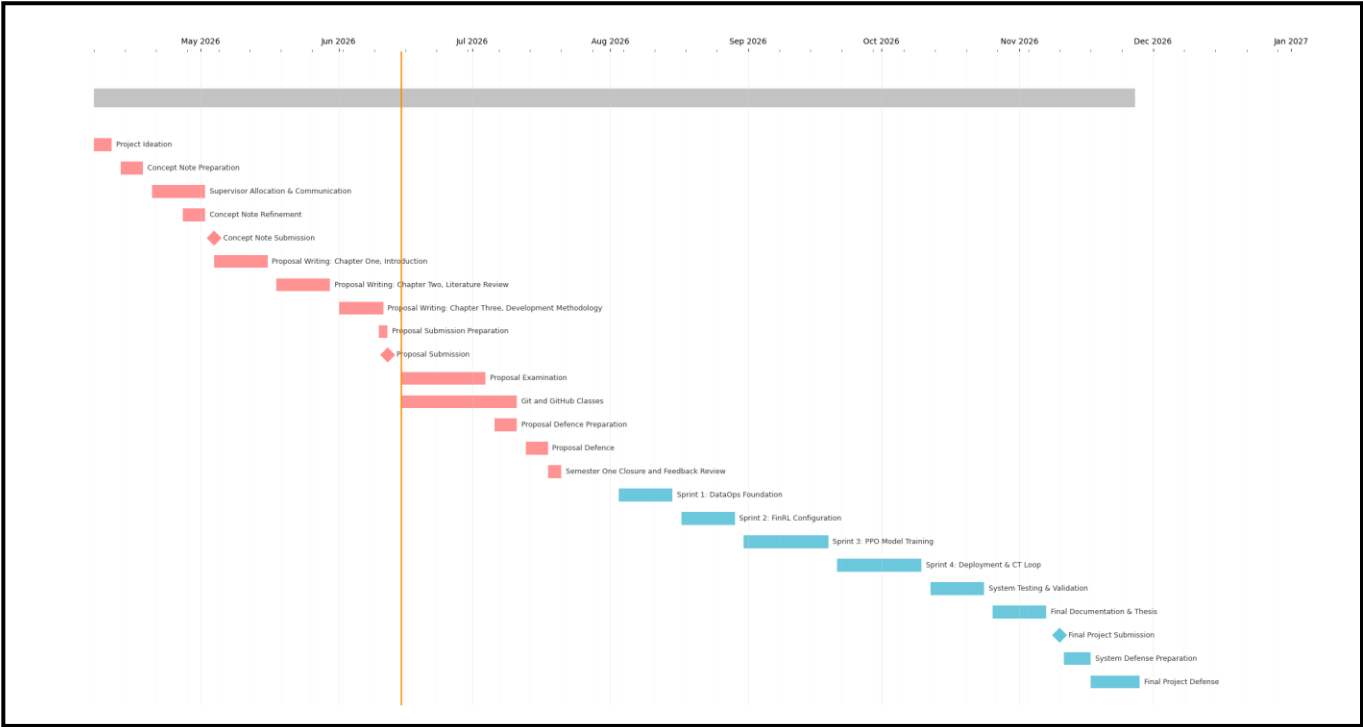
- International Renewable Energy Agency (IRENA). (2024). *Geopolitics of the energy transition: Energy security*. IRENA Publications. https://www.irena.org/-/media/Files/IRENA/Agency/Publication/2024/Apr/IRENA_Geopolitics_transition_energy_security_2024.pdf
- Kreuzberger, D., Kühl, N., & Hirschl, S. (2023a). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, *11*, 31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>
- Kreuzberger, D., Kühl, N., & Hirschl, S. (2023b). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, *11*, 31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>
- Li, Z., Liu, X.-Y., Zheng, J., Wang, Z., Walid, A., & Guo, J. (2021a). FinRL-podracers: High performance and scalable deep reinforcement learning for quantitative finance. *Proceedings of the Second ACM International Conference on AI in Finance*, 1–9. <https://doi.org/10.1145/3490354.3494413>
- Li, Z., Liu, X.-Y., Zheng, J., Wang, Z., Walid, A., & Guo, J. (2021b). FinRL-podracers: High performance and scalable deep reinforcement learning for quantitative finance. *Proceedings of the Second ACM International Conference on AI in Finance*, 1–9. <https://doi.org/10.1145/3490354.3494413>
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under Concept Drift: A Review. *IEEE Transactions on Knowledge and Data Engineering*, 1–1. <https://doi.org/10.1109/TKDE.2018.2876857>
- Meng, S., & Chen, X. (2026). *AI-Driven Alpha Decay: Algorithmic Homogenization, Reflexive Signal Erosion, and the Paradox of Intelligent Markets* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2605.23905>
- Paetzold - Byhain, K., & Scharold, F. (2024). *Classification of uncertainties in agile development of mechatronic systems*. Human Interaction and Emerging Technologies (IHET 2024). <https://doi.org/10.54941/ahfe1005504>

Verma, N., & Xie, Y. (2025). *Optimizing Day-Ahead Energy Trading with Proximal Policy Optimization and Blockchain* (Version 2). arXiv.
<https://doi.org/10.48550/ARXIV.2508.01888>

Xia, Y., You, P., Wang, T., Liu, F., Qi, H., Wu, X., & Zhang, S. (2026). *Agentic Trading: When LLM Agents Meet Financial Markets* (Version 1). arXiv.
<https://doi.org/10.48550/ARXIV.2605.19337>

Appendices

Appendix 1 : Gantt chart



Appendix 2 : Turnitin Report

Page 2 of 42 - Integrity OverviewSubmission ID trn:oid::2945:392998341

19% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **90 Not Cited or Quoted 18%**
Matches with neither in-text citation nor quotation marks
-  **11 Missing Quotations 1%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 12%  Internet sources
- 7%  Publications
- 17%  Submitted works (Student Papers)